ENGINEERING DOCUMENTATION / 03

Database Schema Explanation

Relational design, invariants and transaction flow

A table-by-table explanation of the implemented PostgreSQL schema and the reasoning that protects expense history and settlement correctness.

Document reference	Value
Project	Expense Sharing App
Prepared	16 October 2026
Code baseline	c5a0a64
API / schema	0.1.0 / Alembic head 0002
Currency / default timezone	INR / Asia/Kolkata

Inside this guide

- Entity relationships and storage choices
- Column dictionary and constraints
- Worked accounting example
- Write concurrency and report consistency
- Indexes, migrations and tradeoffs

https://github.com/manansethi1205/expense-sharing-app

Examples use synthetic data. IDs shown in worked examples are illustrative; use IDs returned by your own requests.

Contents

1. Why this relational design 3

2. Entity relationships 4

3. Table: users 5

4. Table: groups 6

5. Table: group_members 7

6. Table: expenses 8

7. Table: expense_shares 9

8. Table: settlements 10

9. Worked storage and accounting example 11

10. Transaction and code flow 12

11. Indexes and query design 13

12. Migrations and future evolution 14

1. Why this relational design

PostgreSQL is the authoritative store. SQLAlchemy 2 maps the schema and issues explicit SQL; Alembic owns schema evolution. A single FastAPI service keeps the assignment understandable and avoids distributed coordination.

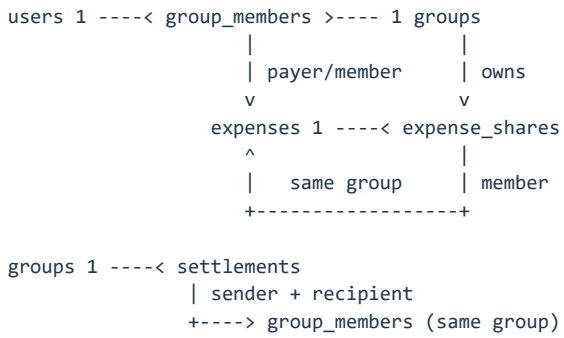
The product can be drawn as a debt graph, but its permanent facts are users, memberships, bills, allocations and payments. Foreign keys, atomic multi-row writes and SQL aggregation fit those facts directly. A graph database would add operational cost without a demonstrated traversal workload, and it falls outside the assignment's suggested SQL options.

Choice	Benefit	Tradeoff
PostgreSQL	Strong constraints, row locks, repeatable-read reports and aggregate queries.	Requires a database service; Docker makes the demo reproducible.
Integer paise	Exact conservation and comparisons; no binary float error.	API must validate decimals and convert at boundaries.
Saved expense shares	Historical obligations survive membership changes and support custom splits.	Stores one row per participant per expense.
Derived balances	One source of truth; no separate mutable balance cache to reconcile.	Read cost grows with group history; optimize only when measured.
Composite membership keys	Database-enforced payer/participant membership within the same group.	More verbose foreign keys and one intentionally repeated group_id.

Core distinction

Expenses record money paid to a merchant. Shares record who consumed that expense. Settlements record money sent between members. Keeping these facts separate prevents repayments from being counted as new spending.

2. Entity relationships



Relationship	Cardinality / enforcement
users to groups	Many-to-many through group_members. Composite PK prevents duplicate memberships.
group to expenses	One-to-many. Every expense has a valid group and a payer in that group.
expense to shares	One-to-many. Each participant has one saved allocation per expense.
share to membership	A participant must belong to the expense's group, enforced through composite foreign keys.
group to settlements	One-to-many. Sender and recipient must both have a membership in that group.

There are six domain tables. alembic_version is migration bookkeeping, not a business entity. SQLAlchemy metadata is used to generate the following column dictionary; the migration scripts are the deployment source of truth.

Deletion policy

Foreign keys use restrictive/default NO ACTION behavior. There are no cascading delete rules and no public delete/member-removal endpoints. This protects historical financial references. User and group names need not be unique; IDs are stable identity.

3. Table: users

A person's stable identity. The service has no login credentials or authentication model. A nonblank name check exists in the database; the API additionally trims and validates its maximum length.

Column	PostgreSQL type	Properties
id	INTEGER	PK; NOT NULL
name	VARCHAR(100)	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL; ORM default

Constraints

Rule	Definition
user_name_nonblank	length(trim(name)) > 0

4. Table: groups

A shared accounting boundary. Currency is constrained to INR. timezone interprets local reporting dates; actual timestamps remain offset-aware instants. Currency/timezone defaults are Python ORM defaults, not SQL server defaults.

Column	PostgreSQL type	Properties
id	INTEGER	PK; NOT NULL
name	VARCHAR(100)	NOT NULL
currency	VARCHAR(3)	NOT NULL; ORM default
timezone	VARCHAR(64)	NOT NULL; ORM default
created_at	TIMESTAMPTZ	NOT NULL; ORM default

Constraints

Rule	Definition
group_inr_only	currency = 'INR'
group_name_nonblank	length(trim(name)) > 0

5. Table: group_members

The current user-to-group association. (group_id, user_id) is both identity and the target of financial composite foreign keys. The API adds memberships but never removes them. joined_at records when the membership was saved.

Column	PostgreSQL type	Properties
group_id	INTEGER	PK; NOT NULL
user_id	INTEGER	PK; NOT NULL
joined_at	TIMESTAMPTZ	NOT NULL; ORM default

Constraints

Rule	Definition
FOREIGN KEY	(group_id) -> (groups.id)
FOREIGN KEY	(user_id) -> (users.id)

Additional indexes

Index	Columns, in order
ix_member_user_group	user_id, group_id

6. Table: expenses

One merchant bill, with payer, amount and immutable allocation mode. category_source and category_rule retain categorization provenance. occurred_at is the business time; created_at is recording time. The extra unique (id, group_id) key lets shares enforce that their group matches the expense.

Column	PostgreSQL type	Properties
id	INTEGER	PK; NOT NULL
group_id	INTEGER	NOT NULL
paid_by	INTEGER	NOT NULL
amount_minor	BIGINT	NOT NULL
description	VARCHAR(500)	NOT NULL
split_type	VARCHAR(10)	NOT NULL
category	VARCHAR(20)	NOT NULL
category_source	VARCHAR(10)	NOT NULL
category_rule	VARCHAR(200)	nullable
occurred_at	TIMESTAMPTZ	NOT NULL; ORM default
created_at	TIMESTAMPTZ	NOT NULL; ORM default

Constraints

Rule	Definition
FOREIGN KEY	(group_id) -> (groups.id)
FOREIGN KEY	(group_id, paid_by) -> (group_members.group_id, group_members.user_id)
expense_amount	amount_minor > 0 AND amount_minor <= 1000000000
expense_category	category IN ('travel','food','accommodation','activities','shopping','other')
expense_category_source	category_source IN ('manual','rule','fallback')
expense_split_type	split_type IN ('equal', 'custom')
uq_expense_id_group	id, group_id

Additional indexes

Index	Columns, in order
ix_expense_group_date	group_id, occurred_at, id
ix_expense_group_payer	group_id, paid_by

7. Table: expense_shares

A snapshot of the amount allocated to each member when an expense is recorded. The composite primary key prevents duplicate participants. group_id is deliberately repeated so foreign keys can prove that expense and participant belong to the same group. A payer's own share is stored as consumption, but excluded from self-debt.

Column	PostgreSQL type	Properties
expense_id	INTEGER	PK; NOT NULL
group_id	INTEGER	NOT NULL
user_id	INTEGER	PK; NOT NULL
amount_minor	BIGINT	NOT NULL

Constraints

Rule	Definition
FOREIGN KEY	(expense_id, group_id) -> (expenses.id, expenses.group_id)
FOREIGN KEY	(group_id, user_id) -> (group_members.group_id, group_members.user_id)
share_nonnegative	amount_minor >= 0

Additional indexes

Index	Columns, in order
ix_share_user_expense	user_id, expense_id

Aggregate invariant

The database checks each share is nonnegative and validates its references. The service, inside the expense transaction, checks complete member coverage and sum(shares)=expense amount. There is no database trigger enforcing that cross-row sum; arbitrary SQL that bypasses the service could violate it.

8. Table: settlements

An immutable record of an external member-to-member payment. mode records which validation semantics were used. It does not move money or contain payment credentials. The wider BIGINT amount range supports paying accumulated debt across multiple individually capped expenses.

Column	PostgreSQL type	Properties
id	INTEGER	PK; NOT NULL
group_id	INTEGER	NOT NULL
from_user_id	INTEGER	NOT NULL
to_user_id	INTEGER	NOT NULL
amount_minor	BIGINT	NOT NULL
mode	VARCHAR(12)	NOT NULL
note	VARCHAR(500)	nullable
occurred_at	TIMESTAMPTZ	NOT NULL; ORM default
created_at	TIMESTAMPTZ	NOT NULL; ORM default

Constraints

Rule	Definition
FOREIGN KEY	(group_id) -> (groups.id)
FOREIGN KEY	(group_id, from_user_id) -> (group_members.group_id, group_members.user_id)
FOREIGN KEY	(group_id, to_user_id) -> (group_members.group_id, group_members.user_id)
settlement_amount	amount_minor > 0
settlement_distinct_users	from_user_id <> to_user_id
settlement_mode	mode IN ('direct','simplified')

Additional indexes

Index	Columns, in order
ix_settlement_group_date	group_id, occurred_at, id

The database rejects self-payments and nonpositive amounts. Current debt direction and overpayment checks require the live ledger and are enforced by the transactional service. The database's BIGINT type provides the upper storage limit; request validation checks that range before insertion.

9. Worked storage and accounting example

Assume users 1=Mohit, 2=Rahul and 3=Ankit belong to group 1=Goa Trip. Values in the storage tables below are paise, not rupees.

Table	Saved rows / important values
group_members	(group=1,user=1), (1,2), (1,3)
expenses	id=10, group_id=1, paid_by=1, amount_minor=120000, description=Dinner, split_type=equal
expense_shares	(expense=10,group=1,user=1,amount=40000); same for users 2 and 3
settlements initially	No rows

Direct obligations are derived from non-payer shares: user 2 owes user 1 40000 paise, and user 3 owes user 1 40000 paise. Mohit's own share exists for analytics, but does not create an edge back to himself.

User	Paid	Allocated	Net before repayment
Mohit	1200.00	400.00	+800.00
Rahul	0.00	400.00	-400.00
Ankit	0.00	400.00	-400.00

Recording Rahul's external payment adds a settlements row (group=1, from=2, to=1, amount_minor=40000, mode=simplified). It does not modify the original expense or shares. Mohit's net becomes +400.00 and Rahul's becomes zero. The second payment clears Ankit and the group.

Membership changes and precision

Adding user 4 later leaves expense 10 with exactly three share rows. A new expense would include four members. For an amount not divisible by member count, integer quotient/remainder allocation gives extra paise to ascending user IDs and preserves the total exactly. Custom splitting stores the validated supplied allocations in the same table.

10. Transaction and code flow

Write path

```
HTTP route -> Pydantic input validation
           -> application service
           -> lock group row FOR UPDATE
           -> read current members / current ledger
           -> validate and compute exact paise results
           -> insert expense + all shares, or settlement
           -> commit once -> serialize response
```

Membership additions and financial writes acquire the same group lock. An expense therefore cannot observe half of a concurrent membership change. Two payments serialize before rechecking debt, preventing duplicate consumption of the same payable balance. The lock is local to one group, so independent groups can progress concurrently.

Balance reads

Normal balance reads coordinate with group writes through a shared group lock and derive the ledger from a single UNION-based query. Opposite pair debts/payments are netted for direct view; signed totals produce net view. Settlement suggestions reuse the derived ledger rather than storing a second financial truth.

Analytics read path

```
analytics route -> report-session dependency
                -> REPEATABLE READ + READ ONLY transaction
                -> SQL CTEs / sums / grouped breakdowns
                -> one consistent report -> close transaction
```

No group share lock is acquired by analytics. PostgreSQL MVCC lets totals, breakdowns and largest expenses see the same snapshot while concurrent expense recording proceeds. Reports aggregate in SQL and expose a paginated drill-down instead of loading the full group history into application memory.

> **Boundary**
>
> Correctness assumes financial writes use this service's locking/validation protocol. Database constraints defend row-level structure, while service rules defend ledger-dependent and cross-row behavior. This distinction is tested and documented rather than hidden behind a claim that every rule is a CHECK constraint.

11. Indexes and query design

Access pattern	Supporting design
Groups for a user	group_members(user_id, group_id) reverses the composite PK's leading group order.
Expense history by group/date	expenses(group_id, occurred_at, id) supports date ranges and stable ties.
Bills paid by a member	expenses(group_id, paid_by) supports payer filtering.
Personal consumption	expense_shares(user_id, expense_id) supports share-based joins and reports.
Payment history	settlements(group_id, occurred_at, id) supports group/date access.
Referential joins	Primary keys and expense (id, group_id) uniqueness support referenced-key lookups.

An index's existence does not guarantee every combined filter or sort avoids a scan. Substring search uses escaped case-insensitive matching; there is no trigram/full-text index in this demo. Monthly/category reports use SQL aggregation over filtered rows. Measure EXPLAIN ANALYZE and dataset size before adding indexes, materialized summaries or a balance cache.

Avoiding overengineering

Category values are a small fixed vocabulary enforced with text checks, not a separate editable category service. Settlement plans are computed previews, not persisted plans that become stale. A balance table, graph store, queue and microservices would increase synchronization work without improving the assignment's core guarantees.

12. Migrations and future evolution

Revision	Change
0001	Initial six-table schema, constraints and indexes. The original settlement amount check included the per-expense cap.
0002 (head)	Replaces the settlement amount check with positivity only, allowing aggregate payments up to BIGINT storage capacity. Expense limits stay unchanged.

```
uv run alembic upgrade head
uv run alembic current
uv run alembic check
```

The Compose migration service upgrades before API startup. The app does not call metadata.create_all in normal operation. ORM UTC/default values are applied when the app creates rows; a raw SQL insertion must supply required fields without server defaults.

Downgrading 0002 would restore the old settlement cap. If larger payments already exist, the downgrade fails atomically rather than deleting or rewriting those records. Take a backup and inspect data compatibility before attempting any downgrade.

Tests and evidence

Integration tests migrate isolated PostgreSQL schemas and verify constraints, snapshot shares, aggregate-payment migration behavior, concurrent settlement rejection and reports that remain consistent without blocking writers. Unit tests cover money arithmetic, zero-sum conservation, category rules and exact optimization. The current baseline has 83 passing tests.

With more time

Add authenticated group authorization, idempotency keys for recording, an explicit reversal/correction model, and measured query optimization. Multi-currency support needs currency-specific minor units and a clear exchange-rate policy; merely removing the INR check would be insufficient. Membership removal needs historical validity rules because existing financial references must remain intact.

Implementation references: app/models.py, migration revisions, decisions.md and flow.md. The version stamp identifies the code revision this guide explains.